```
            new EmailServerSettings                    ❺
            (                                          ❺
                Host: "smtp.server.com",               ❺
                Port: 25                               ❺
            ))                                         ❺
        );
    emailSender.SendEmail(username);                   ❻
    return $"Email sent to {username}!";
}
```

❶ The endpoint is called when a new user is created.

❷ To create EmailSender, you must create all its dependencies.

❸ You need a new MessageFactory.

❹ The NetworkClient also has dependencies.

❺ You're already two layers deep, but there could feasibly be more.

❻ Finally, you can send the email.

We subsequently refactored this code to inject an instance of `IEmailSender` into the handler instead, as shown in listing 9.2. The `IEmailSender` interface decouples the endpoint handler from the `EmailSender` implementation, making it easier to change the implementation of `EmailSender` (or replace it) without having to rewrite `RegisterUser`.

## Listing 9.2 Using `IEmailSender` with dependency injection in an endpoint handler

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapGet("/register/{username}", RegisterUser);              ❶

app.Run();

string RegisterUser(string username, IEmailSender emailSender)   ❷
{
    emailSender.SendEmail(username);                           ❸
    return $"Email sent to {username}!";
}
```